

Diziler, Matrisler ve Bağlı Listeler

CEN207 Veri Yapıları — Hafta 2

Dr. Öğr. Üyesi Uğur CORUH · 2026-2027 Güz

Bugünün planı (3 saat)

Saat	Konu
1	Diziler: ekleme/silme, dinamik büyüme Anim 1–2 · 2B yerleşim, döndürme, yeniden düzenleme Anim 3–5
2	Seyrek matrisler Anim 6–8 · listeler: düğüm/baş/NULL, tekil işlemler Anim 9–12
3	Çift yönlü, dairesel, Josephus, XOR, atlamalı liste Anim 13–17 · diziler ve listeler

Öğrenme çıktıları: ÖÇ.1 (temel veri yapılarını açıklama) · ÖÇ.2 (karmaşıklık analizi) · ÖÇ.7 (doğru yapıyı seçme)

Bu haftanın kavramları — nerede

Kavram	Nerede
Diziler: kaydırma, dinamik büyüme	Bölüm 1
2B diziler, döndürme, yeniden düzenleme	Bölüm 2–3
Seyrek matrisler	Bölüm 4
Tekil/çift yönlü/dairesel listeler	Bölüm 5–8
XOR liste, atlamalı liste, diziler ve listeler	Bölüm 9–11

Kod örnekleri nasıl çalışıyor

- Her fikrin eksiksiz bir **C** ve **Java** programı var
- C: `gcc -std=c11 -Wall -Wextra -o /tmp/x file.c && /tmp/x`
- Java: `javac -d /tmp/j File.java && java -cp /tmp/j File`
- Kaynaklar: `code/week-02/c/` ve `code/week-02/java/`
- Her programın beklenen çıktısı hafta notunda var

Hatırlatma — Hafta 1: bellek ve işaretçiler

- Bir değişken, bellekte etiketli bir **kutu**dur
- Bir işaretçi (pointer) bir **adres saklar** — bir kutuyu gösterir
- `malloc(n)` : `n` bayt ayırır, bir adres döndürür
- `free(p)` : kutuyu geri verir; sonra `p = NULL` yapın
- `NULL` : **hiçbir şeyi** göstermeyen bir işaretçi

Haftanın haritası — genel bakış

Diziler ve matrisler	Bağlı listeler
Kaydırma, dinamik büyüme	Düğüm, baş, NULL
Satır öncelikli, döndürme, düzenleme	Tekil, çift yönlü, dairesel
Seyrek: triplet, devrik, toplama	Josephus, XOR, atlamalı liste

1. Bellekte Diziler: Ekleme, Silme, Büyüme

Bir soru ile başlayalım

Bir dizi, art arda n kutudur. Ortasına bir değer eklendiğinde, yeni değer yerine oturmadan önce ne hareket etmek zorunda?

Sezgi — numaralı dolaplardan oluşan bir sıra

- Her kutu, başlangıçtan **indis** × **kutu boyutu** uzaklıkta
- `arr[k]` saf aritmetik — arama gerekmez
- Ama dolaplar kendi kendine kayıp ayrılmaz
- Yer açmak, ondan sonraki her dolabı hareket ettirmek demek

size ve kapasite

- Sabit bir dizi baştan CAP kutu ayırır
- size, **gerçekte** kaç kutunun kullanıldığını takip eder
- insert_at(k, v) : k indisinde yer aç
- delete_at(k) : k indisindeki boşluğu kapat

Dizide ekleme ve silme, adım adım



Bitti: size = 9, toplam 24 kaydırma. $k = 0$ en yavaş ($O(n)$), $k = \text{size}$ en hızlı ($O(1)$) eklemedir.

Uç durum — taşma: dizi dolu



Bitti: size = 10, toplam 12 kaydırma, 2 taşma. $k = 0$ en yavaş ($O(n)$), $k = \text{size}$ en hızlı ($O(1)$) eklemedir.

Kod — insert_at()

```
bool insert_at(int k, int v) {  
    if (size == CAP)  
        return false;          /* full: overflow */  
    for (int i = size; i > k; i--)  
        arr[i] = arr[i - 1];    /* shift right */  
    arr[k] = v;  
    size++;  
    return true;  
}
```

Kod — delete_at()

```
bool delete_at(int k) {  
    if (size == 0)  
        return false;    /* empty: underflow */  
    for (int i = k; i < size - 1; i++)  
        arr[i] = arr[i + 1]; /* shift left */  
    size--;  
    return true;  
}
```

Karmaşıklık ve sık yapılan hatalar

- $k = 0$: **her** elemanı kaydırır — $O(n)$
- $k = \text{size}$ (sona ekleme): hiç kaydırma yok — $O(1)$
- Hata: önce taşma/alttan taşma kontrolünü unutmak
- Hata: yanlış yönde kaydırıp veriyi ezmek

Mini soru

5 değeri hep 0. indise eklediniz.
Toplam kaç eleman-kaydırması oldu?

Yanıt

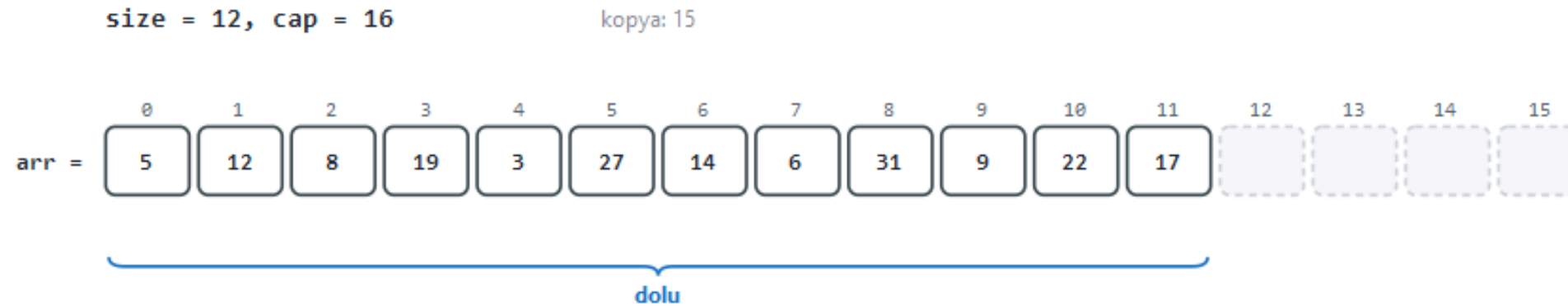
$0+1+2+3+4 = 10$ kaydırma. Her baştan ekleme, o an var olan her elemanı kaydırır — klasik $O(n)$ en kötü durum, beş kez tekrarlanmış.

Peki ya boyut önceden bilinmiyorsa?

Sabit bir dizinin **CAP** 'i katı bir tavadır.

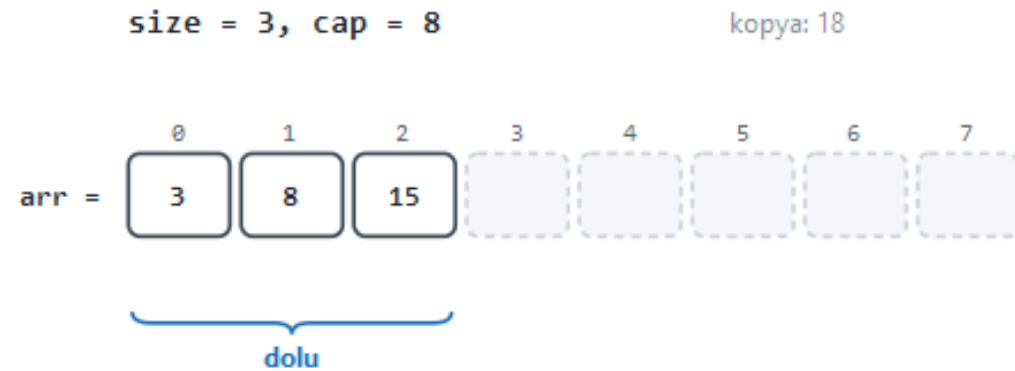
Bir **dinamik dizi**, eklemeyi reddetmek yerine kendi kapasitesini talep üzerine büyütür.

Dinamik dizi büyümesi, adım adım



Bitti: 12 ekleme, 4 büyüme, toplam 15 kopya ($2 \cdot 12 = 24$ 'den az). Bir ekleme bazen $O(n)$ sürer ama ORTALAMASI (amortized) $O(1)$ 'dir.

Uç durum — geri küçülme



Bitti: 12 ekleme, 3 büyüme, 1 küçülme, toplam 18 kopya ($2 \cdot 12 = 24$ 'den az). Bir ekleme bazen $O(n)$ sürer ama ORTALAMASI (amortized) $O(1)$ 'dir.

Kod — `da_resize()`

```
static void da_resize(DynArray *a, int new_cap) {
    int *fresh = malloc(new_cap * sizeof(int));
    for (int i = 0; i < a->size; i++)
        fresh[i] = a->data[i];    /* copy every value */
    free(a->data);                /* old block freed */
    a->data = fresh;
    a->cap = new_cap;
}
```

Kod — da_append()

```
void da_append(DynArray *a, int v) {  
    if (a->size == a->cap) {  
        int new_cap = a->cap * 2;    /* growth factor 2 */  
        da_resize(a, new_cap);  
    }  
    a->data[a->size++] = v;  
}
```

Karmaşıklık ve sık yapılan hatalar

- Büyüyen bir ekleme `size` elemanı kopyalar: $O(n)$
- Çoğu ekleme yalnızca bir hücre yazar: $O(1)$
- `n` eklemede toplam kopya sayısı $2n$ 'nin altında kalır
- Yani ekleme başına **amortize edilmiş** maliyet $O(1)$ 'dir
- Hata: her tek eklemeyi $O(n)$ sanmak

Mini soru

`cap = 1` 'den başlayıp ikiye katlayarak, 100 değer teker teker eklenirken dizi kaç kez büyür?

Yanıt

7 kez ($1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rightarrow 64 \rightarrow 128$).

$\log_2(100) \approx 6.6$, yukarı yuvarlanmış — büyüme son boyuta göre **logaritmiktir**, doğrusal değil.

2. İki Boyutlu Diziler: Satır Öncelikli mi, Sütun Öncelikli mi?

Bir soru ile başlayalım

Bellek, baytlardan oluşan tek uzun bir sıra.
Bir matris iki boyutlu görünür — satır ve
sütun. `mat[i][j]` nasıl tek bir adrese döner?

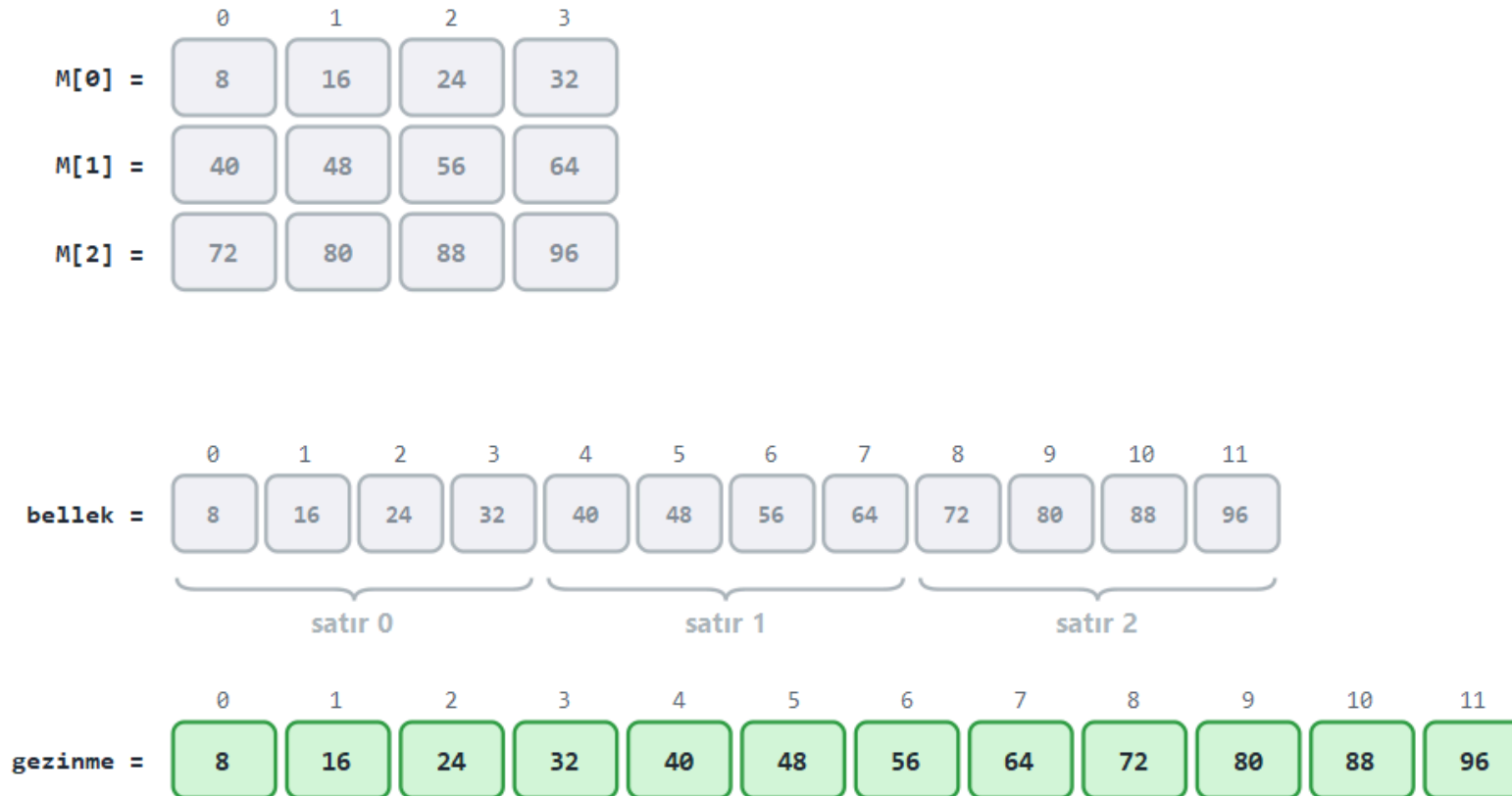
Sezgi — bir kitaplık, bir raf sırayla

- Matrisin satırlarını uç uca dizildiğini hayal edin
- Önce 0. satırın hücreleri, sonra 1. satırınki, böyle devam
- Bu yerleşime **satır öncelikli (row-major)** denir
- Bazı diller (Fortran, MATLAB) önce **sütunları** dizer

Adres formülleri

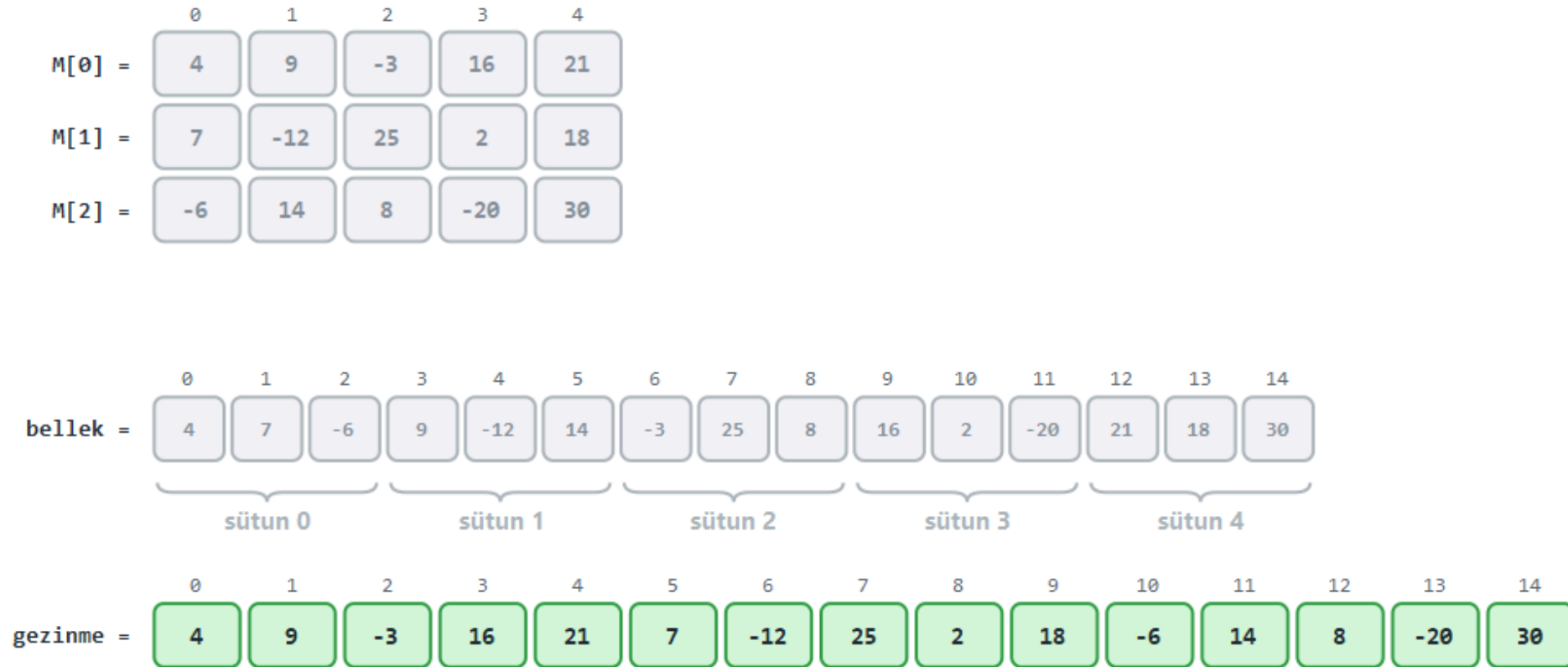
- **Satır öncelikli:** $\text{addr}(i,j) = i * \text{COLS} + j$
- **Sütun öncelikli:** $\text{addr}(i,j) = j * \text{ROWS} + i$
- C ve Java: satır öncelikli; Fortran, MATLAB: sütun öncelikli
- Formül aritmetik — hiç arama yok

Bellekte matris, adım adım



Bitti: 12 hücre, 11/11 adım bitişik ($\Delta=1$), en büyük atlama $\Delta=1$. Gezinme depolamayla EŞLEŞTİĞİ için ön bellek dostu.

Uç durum — sütun öncelikli depolama, satır satır gezinme



Bitti: 15 hücre, 0/14 adım bitişik ($\Delta=1$), en büyük atlama $\Delta=11$. Gezinme depolamayla ters düştüğü için ön bellek İÇİN KÖTÜ.

Kod — `addr()` ve satır öncelikli bir gezinme

```
int mat[ROWS][COLS];  
/* address of mat[i][j], in ints from the start */  
int addr(int i, int j) {  
    return i * COLS + j;    /* row-major */  
}  
  
void traverse_row_major(int mat[ROWS][COLS]) {  
    for (int i = 0; i < ROWS; i++)  
        for (int j = 0; j < COLS; j++)  
            visit(mat[i][j]);  
}
```

Karmaşıklık ve sık yapılan hatalar

- Adres hesabı: saf aritmetik — $O(1)$
- Satır öncelikli gezinme, satır öncelikli depolama: her adım $\Delta=1$
- Gezinmeyle uyuşmayan yerleşim: **her** adımda bir sıçrama
- Hata: her dilin satır öncelikli olduğunu varsaymak

Mini soru

Satır öncelikli bir matrisin 5 sütunu var.

`mat[3][2]` adres 17'de. `mat[3][3]` 'ün adresi ne?

Yanıt

18. Bir sütun sağa gitmek, satır öncelikli düzende tam olarak **1** ekler — bir sonraki hücre her zaman bir sonraki adrestir.

3. Döndürme ve Yeniden Düzenleme: İki İşaretçi

Bir soru ile başlayalım

12 değerlik bir diziyi, ikinci bir dizi kullanmadan 4 sola döndürün. Yalnızca $O(1)$ ek alan varken nereden başlarsınız?

Sezgi — üç kez ters çevirme

- Önce ilk parçayı ters çevirin, sonra kalanını
- Sonra dizinin **tamamını** bir kez daha ters çevirin
- Üç ters çevirme her değeri tam yerine oturtur
- İkinci dizi yok, tek tek kaydırma yok

rotate_left : üç ters çevirme

- $d = d \% n$ — gerçek döndürme miktarı
- $\text{arr}[0..d-1]$ 'i ters çevir
- $\text{arr}[d..n-1]$ 'i ters çevir
- $\text{arr}[0..n-1]$ 'i ters çevir — bitti

Dizi döndürme, adım adım

A =

0	1	2	3	4	5	6	7	8	9	10	11
50	60	70	80	90	100	110	120	10	20	30	40

reverse(0,3) sonrası

0	1	2	3	4	5	6	7	8	9	10	11
40	30	20	10	50	60	70	80	90	100	110	120

reverse(4,11) sonrası

0	1	2	3	4	5	6	7	8	9	10	11
40	30	20	10	120	110	100	90	80	70	60	50

reverse(0,11) sonrası = final

0	1	2	3	4	5	6	7	8	9	10	11
50	60	70	80	90	100	110	120	10	20	30	40

Bitti: d=4 sola döndürme, 12 takas ile $O(n)$ sürede. Sonuç: 50, 60, 70, 80, 90, 100, 110, 120, 10, 20, 30, 40.

Uç durum — n'den büyük d

A =

0	1	2	3	4	5	6	7	8	9
2	28	9	35	16	4	22	7	14	21

reverse(0,2) sonrası

0	1	2	3	4	5	6	7	8	9
21	14	7	2	28	9	35	16	4	22

reverse(3,9) sonrası

0	1	2	3	4	5	6	7	8	9
21	14	7	22	4	16	35	9	28	2

reverse(0,9) sonrası = final

0	1	2	3	4	5	6	7	8	9
2	28	9	35	16	4	22	7	14	21

Bitti: d=23 sola döndürme, 9 takas ile O(n) sürede. Sonuç: 2, 28, 9, 35, 16, 4, 22, 7, 14, 21.

Kod — reverse()

```
void reverse(int arr[], int lo, int hi) {  
    while (lo < hi) {  
        int tmp = arr[lo];  
        arr[lo] = arr[hi];  
        arr[hi] = tmp;  
        lo++;  
        hi--;  
    }  
}
```

Kod — rotate_left()

```
void rotate_left(int arr[], int n, int d) {  
    if (n == 0) return;          /* empty: nothing to do */  
    d = d % n;  
    reverse(arr, 0, d - 1);      /* first d */  
    reverse(arr, d, n - 1);      /* the rest */  
    reverse(arr, 0, n - 1);      /* whole array */  
}
```

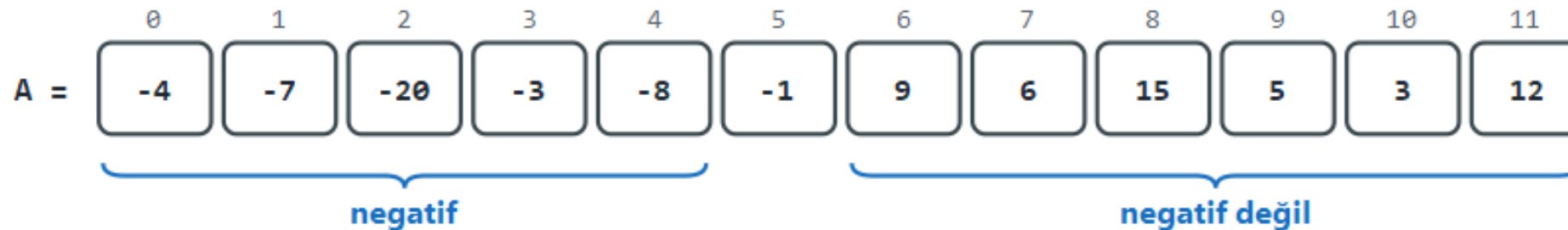
Karmaşıklık ve sık yapılan hatalar

- Dizi üzerinde üç geçiş, her biri $O(n)$ — yine de $O(n)$
- $O(1)$ ek alan: hiçbir yerde ikinci bir dizi yok
- Hata: önce `d = d % n` 'i unutmak
- Hata: `reverse` içinde birer-fazla sınır hatası

Yeniden düzenleme: iki işaretçi birbirine yaklaşır

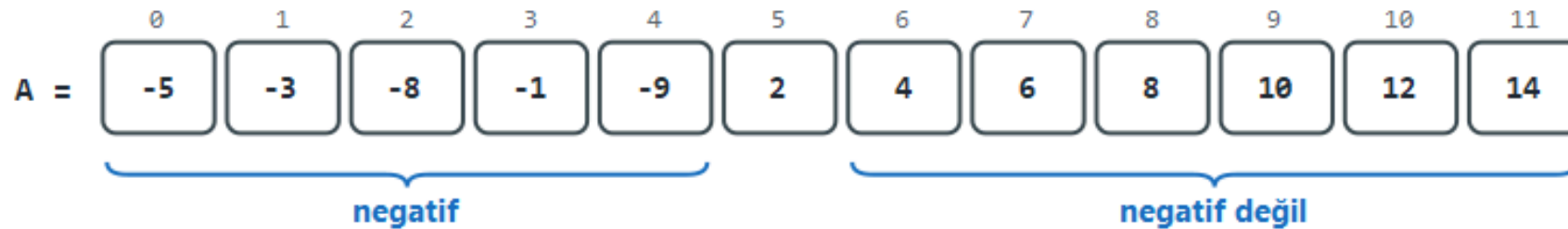
- Amaç: negatifler solda, negatif olmayanlar sağda
- `left`, zaten negatif olan değerleri atlar
- `right`, zaten negatif olmayan değerleri atlar
- İkisi de durunca, takas edip içe doğru ilerleyin

Diziyi yeniden düzenleme, adım adım



Bitti: 3 takas ile negatifler sola, negatif olmayanlar sağa toplandı: -4, -7, -20, -3, -8, -1, 9, 6, 15, 5, 3, 12. Tek geçişte $O(n)$.

Uç durum — zaten ayrılmış



Bitti: 0 takas ile negatifler sola, negatif olmayanlar sağa toplandı: -5, -3, -8, -1, -9, 2, 4, 6, 8, 10, 12, 14. Tek geçişte $O(n)$.

Kod — segregate()

```
void segregate(int arr[], int n) {  
    int left = 0, right = n - 1;  
    while (left < right) {  
        while (left < right && arr[left] < 0) left++;  
        while (left < right && arr[right] >= 0) right--;  
        if (left < right) {  
            int tmp = arr[left];  
            arr[left] = arr[right];  
            arr[right] = tmp;  
            left++; right--;  
        }  
    }  
}
```

Karmaşıklık ve sık yapılan hatalar

- İki işaretçi de yalnızca **içe doğru** ilerler — toplam $O(n)$
- Tek geçiş, $O(1)$ ek alan
- Hata: `0` 'ı negatif saymak — **değildir**
- Hata: iç döngülerde `left < right` korumasını unutmak

Mini soru

segregate 'ten sonra dizi her yarı içinde
sıralı mı, yoksa yalnızca ikiye mi ayrılmış?

Yanıt

Yalnızca ayrılmış. Negatifler negatif olmayanların soluna geçer, ama hiçbir yarı sıralı değildir — segregate sıralamak için değerleri hiç karşılaştırmaz.

4. Seyrek Matrisler

Bir soru ile başlayalım

1000×1000'lik bir matrisin bir milyon hücresi var. Yalnızca 200'ü sıfır değil. Diğer 999.800 sıfırı neden saklayasınız ki?

Sezgi — büyük ölçüde boş bir otopark

- Çoğu yer, gün boyu, her gün boş durur
- Yalnızca **dolu** yerleri listeleyen bir pano yeterlidir
- Konum + değer, hatırlanması gereken her şey budur
- O pano, **triplet** gösterimidir

Triplet: (row, col, value)

- Her **sıfır olmayan** hücre için bir kayıt, başka hiçbir şey yok
- Satır öncelikli tara, her sıfırı atla
- Bir seyrek matrisin triplet tablosu çok küçük olabilir
- Yoğun matrisi geri kurmak yalnızca triplet'leri gerektirir

Seyrek matris triplet olarak, adım adım

	0	1	2	3	4	5
M[0] =	0	0	3	0	0	0
M[1] =	0	0	0	0	4	0
M[2] =	0	5	0	0	0	0
M[3] =	0	0	0	0	0	7
M[4] =	6	0	0	8	0	0

rows=5, cols=6, count=6

row	col	value
0	2	3
1	4	4
2	1	5
3	5	7
4	0	6
4	3	8

Bitti: 30 hücreden yalnız 6 tanesi sıfır değil (%80 sıfıra ayrıldı). Triplet tablosu 6 satır -- yoğun matristen çok daha küçük.

Uç durum — tamamen sıfır bir matris

	0	1	2	3
M[0] =	0	0	0	0
M[1] =	0	0	0	0
M[2] =	0	0	0	0
M[3] =	0	0	0	0

rows=4, cols=4, count=0

row col value

Biti: 16 hücreden yalnız 0 tanesi sıfır değil (%100 sıfıra ayrılırdı). Triplet tablosu 0 satır -- yoğun matristen çok daha küçük.

Kod — `to_triplets()`

```
int to_triplets(int mat[ROWS][COLS], Triplet out[]) {  
    int k = 0;  
    for (int i = 0; i < ROWS; i++)  
        for (int j = 0; j < COLS; j++)  
            if (mat[i][j] != 0) {  
                out[k].row = i;  
                out[k].col = j;  
                out[k].value = mat[i][j];  
                k++;  
            }  
    return k;  
}
```

Karmaşıklık

- Yoğun matrisi tarama: $O(\text{satır} \times \text{sütun})$
- Çıktı boyutu, **sıfır olmayan sayısına** eşit, asla fazla değil
- Çok seyrek bir matriste: triplet'ler çok daha küçük

Hızlı devrik: yeniden sıralama gerekmez

- Basit devrik: `(row, col)` 'u takas et, sonra **sırala** — $O(\text{nnz} \log \text{nnz})$
- Daha iyisi: önce sütun başına sıfır olmayanları **say**
- Sayıları başlangıç **konumlarına** çevir (örnek toplamı)
- Bir geçiş daha, her triplet'i zaten sıralı yerleştirir

Hızlı devrik, adım adım

	0	1	2	3	4
M[0] =	0	0	3	0	4
M[1] =	0	0	5	7	0
M[2] =	0	0	0	0	0
M[3] =	6	0	0	0	0

a[] (özgün) =		
row	col	value
0	2	3
0	4	4
1	2	5
1	3	7
3	0	6

	0	1	2	3	4
sayım[] =	1	0	2	1	1

sütun başına sıfır olmayan sayısı

	0	1	2	3	4
konum[] =	0	1	1	3	4

sütun c, b[]'de burada başlar

b[] (devrik) =		
row	col	value
0	3	6
2	0	3
2	1	5
3	1	7
4	0	4

Bitti: 5 üçlü, b[]'ye tek bir ek geçişte ($O(nnz + COLS)$) yerleşti -- genel sıralamadan ($O(nnz \log nnz)$) çok daha hızlı.

Uç durum — tamamen dolu bir matris

	0	1	2
M[0] =	1	2	3
M[1] =	4	5	6
M[2] =	7	8	9
M[3] =	10	11	12

	0	1	2
sayım[] =	4	4	4

sütun başına sıfır olmayan sayısı

	0	1	2
konum[] =	0	4	8

sütun c, b[]'de burada başlar

	0	1	2
b[0] =	0	0	1
b[1] =	0	1	4
b[2] =	0	2	7
b[3] =	0	3	10
b[4] =	1	0	2
b[5] =	1	1	5
b[6] =	1	2	8
b[7] =	1	3	11
b[8] =	2	0	3
b[9] =	2	1	6
b[10] =	2	2	9
b[11] =	2	3	12

a[] (özgün) =		
row	col	value
0	0	1
0	1	2
0	2	3
1	0	4
1	1	5
1	2	6
2	0	7
2	1	8
2	2	9
3	0	10
3	1	11
3	2	12

Bitti: 12 üçlü, b[]'ye tek bir ek geçişte ($O(nnz + COLS)$) yerleşti -- genel sıralamadan ($O(nnz \log nnz)$) çok daha hızlı.

Kod — sayma ve örnek toplamaları

```
int count[COLS] = {0};
int pos[COLS];
for (int i = 0; i < nnz; i++)
    count[a[i].col]++;          /* per column */
pos[0] = 0;
for (int c = 1; c < COLS; c++)
    pos[c] = pos[c - 1] + count[c - 1];
```

Kod — her triplet'i yerleştirme

```
for (int i = 0; i < nnz; i++) {  
    int c = a[i].col;  
    int p = pos[c]++;  
    b[p].row = a[i].col;    /* row/col swap */  
    b[p].col = a[i].row;  
    b[p].value = a[i].value;  
}
```

Karmaşıklık ve sık yapılan hatalar

- İki geçiş, $O(\text{nnz} + \text{COLS})$ — hiç sıralama yok
- Genel bir sıralamanın $O(\text{nnz} \log \text{nnz})$ 'sinden çok daha hızlı
- Hata: önek toplamını atlamak — çıktı sırasız kalır
- Hata: `row` ile `col` 'u takas etmeyi unutmak

İki seyrek matrisi toplama: bir birleştirme

- İki triplet listesi de zaten satır öncelikli **sıralı**
- İkisini birlikte, merge sort'un birleştirme adımı gibi yürü
- Erken gelen `(row, col)` kazanır ve doğrudan kopyalanır
- İkisinde de aynı `(row, col)` : değerleri **topla**

Seyrek matris toplama, adım adım

a[] =

row	col	val
0	0	4
0	3	2
1	1	5
2	0	3
2	2	7
2	3	1

b[] =

row	col	val
0	0	6
0	1	3
1	1	-2
1	3	8
2	1	4
2	3	9

toplam (sum) =

row	col	val
0	0	10
0	1	3
0	3	2
1	1	3
1	3	8
2	0	3
2	1	4
2	2	7
2	3	10

Bitti: 6 + 6 üçlü tek bir birleştirmede (merge) 9 sonuç üçlüsüne indi (3 iptal/örtüşme). O(na + nb), yeniden sıralama yok.

Uç durum — birbirini götüren değerler

a[] =

row	col	val
0	0	5
0	2	3
1	1	4
1	2	-6
2	0	2
2	1	9

b[] =

row	col	val
0	0	-5
0	1	7
1	1	6
1	2	6
2	0	-2
2	2	9

toplam (sum) =

row	col	val
0	1	7
0	2	3
1	1	10
2	1	9
2	2	9

Bitti: 6 + 6 üçlü tek bir birleştirmede (merge) 5 sonuç üçlüsüne indi (7 iptal/örtüşme). $O(na + nb)$, yeniden sıralama yok.

Kod — birleştirme döngüsü

```
while (i < na && j < nb) {  
    if (/* ... a[i]'s (row,col) comes first ... */) {  
        out[k++] = a[i++];  
    } else if (/* ... b[j]'s comes first ... */) {  
        out[k++] = b[j++];  
    } else {  
        int sum = a[i].value + b[j].value;  
        if (sum != 0) /* ... out[k++] = sum entry ... */;  
        i++; j++;  
    }  
}
```

Karmaşıklık ve sık yapılan hatalar

- Tek bir birleştirme geçişi: $O(na + nb)$, yeniden sıralama yok
- **İkisinde de** olmayan bir hücreye hiç dokunmaz
- Hata: toplam 0 olduğunda hücreyi düşürmeyi unutmak
- Hata: sırasız triplet listelerinin yine de birleşeceğini varsaymak

Mini soru

(1,2) hücresi A matrisinde 6, B matrisinde
–6. Toplamın triplet listesinde ne kalır?

Yanıt

Hiçbir şey. $6 + (-6) = 0$, ve toplamı sıfır olan bir hücre tamamen düşürülür — triplet listesi yalnızca sıfır olmayan değerleri tutar.

5. Bağlı Listeler: Düğüm, Baş, NULL

Bir soru ile başlayalım

Bir diziye her ekleme **n** elemana kadarını kaydırabilir. Ekleme yapmanın var olan değerleri **hiçbir zaman** hareket ettirmediği bir yapı var mı?

Kısa bir tarihçe

- **1956** — John McCarthy, Lisp'in **cons hücresi**
- Bir cons hücresi: bir değer, artı bir sonrakine işaretçi
- Altı on yıl sonra hâlâ her dilde bu fikir kullanılıyor
- C'nin `struct Node { data; next; }` 'i aynı fikir

Sezgi — bir define avı

- Her ipucu bir sonraki ipucunun **nerede** olduğunu söyler
- Haritanın tamamını asla bir kerede görmezsiniz
- Bir işaretçiyi izle, var, oku, bir sonrakini izle
- **Son** ipucu "burada bir şey yok" der — `NULL`

Tüm fikir, tek bir struct'ta

```
typedef struct Node {  
    int data;  
    struct Node *next;  
} Node;
```

- Bir değer, bir işaretçi — bağlı liste bu kadar
- Kaydırma yok: yeni bir düğüm yalnızca bir işaretçi alır

head ve NULL sonlandırıcısı

- head , listeye giren tek sabit referanstır
- Her başka düğüme next i izleyerek ulaşılır
- Son düğümün next i NULL dır — "başka düğüm yok"
- Boş bir liste yalnızca head == NULL dır

"Inception": kendini içeren bir struct

- `struct Node` 'un `struct Node *` türünde bir alanı var
- Bu sonsuz özyineleme **değil** — bir işaretçi
- Bir işaretçinin boyutu sabit, `Node` tamamlanmadan bile bilinir
- Bu yüzden `struct Node *next;` derlenir, ama `Node next;` derlenemez

Sık yapılan hatalar

- `->next` 'ten önce `NULL` kontrolünü unutmak
- Bir **düğümü** (struct) ona giden bir **işaretçiyle** karıştırmak
- Bir işaretçi gerekirken `Node next;` yazmak

Mini soru

`next` neden düz bir `struct Node` değil,
mutlaka `struct Node *` olarak tanımlanmalı?

Yanıt

Düz bir `Node` alanı, `Node` 'un **boyutunun** önceden bilinmesini gerektirirdi — ama `Node` henüz tanımlanmayı bitirmedi. Bir işaretçinin boyutu buna hiç bağlı değildir.

6. Tekil Bağlı Listeler: Ekleme, Silme, Arama, Tersine Çevirme

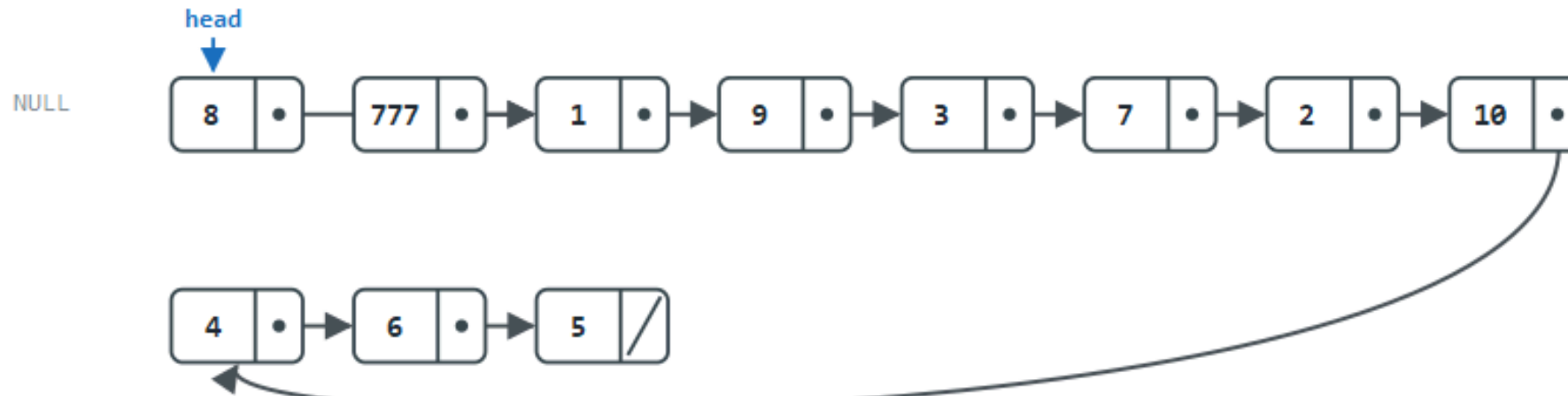
Bir soru ile başlayalım

Üç ekleme konumu — baş, son ve "belirli bir düğümden sonra" — çok farklı bedellere mal oluyor. Hangisi ucuz, hangisi değil?

Eklemenin üç yolu

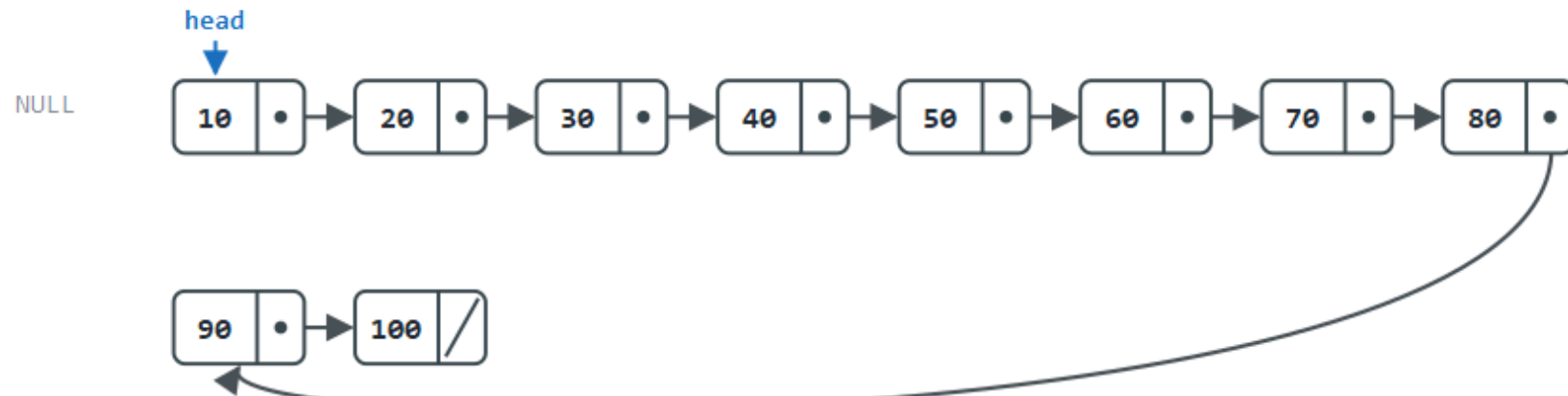
- `insert_head` : yeni düğüm eski başı gösterir — $O(1)$
- `insert_tail` : burada son işaretçisi yok — sona kadar yürü
- `insert_after(prev, v)` : iki işaretçi yazımı, **sırayla**
- Bu sırayı ters çevirmek listeyi **koparır**

Tekil listede ekleme, adım adım



Bitti: 5 baştan, 5 sondan, 1 belirli bir düğümden sonra ekleme. Liste (baştan sona): 8, 777, 1, 9, 3, 7, 2, 10, 4, 6, 5.

Uç durum — iki adım, ters sırayla



Bitti: 0 baştan, 10 sondan, 0 belirli bir düğümden sonra ekleme. Liste (baştan sona): 10, 20, 30, 40, 50, 60, 70, 80, 90, 100.

Kod — insert_head()

```
Node *insert_head(Node *head, int value) {  
    Node *n = malloc(sizeof(Node));  
    n->data = value;  
    n->next = head;    /* points at the old head */  
    return n;          /* new node is head now */  
}
```

Kod — `insert_after()` : sıra önemli

```
void insert_after(Node *prev, int value) {  
    Node *n = malloc(sizeof(Node));  
    n->data = value;  
    n->next = prev->next;    /* STEP 1: new first */  
    prev->next = n;          /* STEP 2: then link */  
}
```

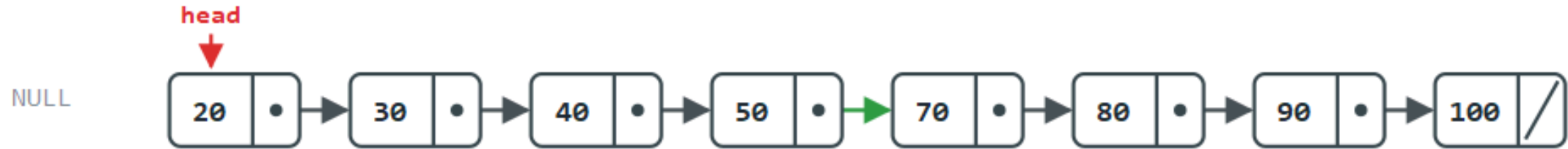
Karmaşıklık ve sık yapılan hatalar

- `insert_head` : $O(1)$ — hiç yürüme yok
- `insert_tail` (son işaretçisi yok): sona kadar yürür — $O(n)$
- `insert_after` : `prev` zaten bulunmuşsa $O(1)$
- Hata: 1. ve 2. ADIM'ı takas etmek — listeyi koparır

Değere göre silme

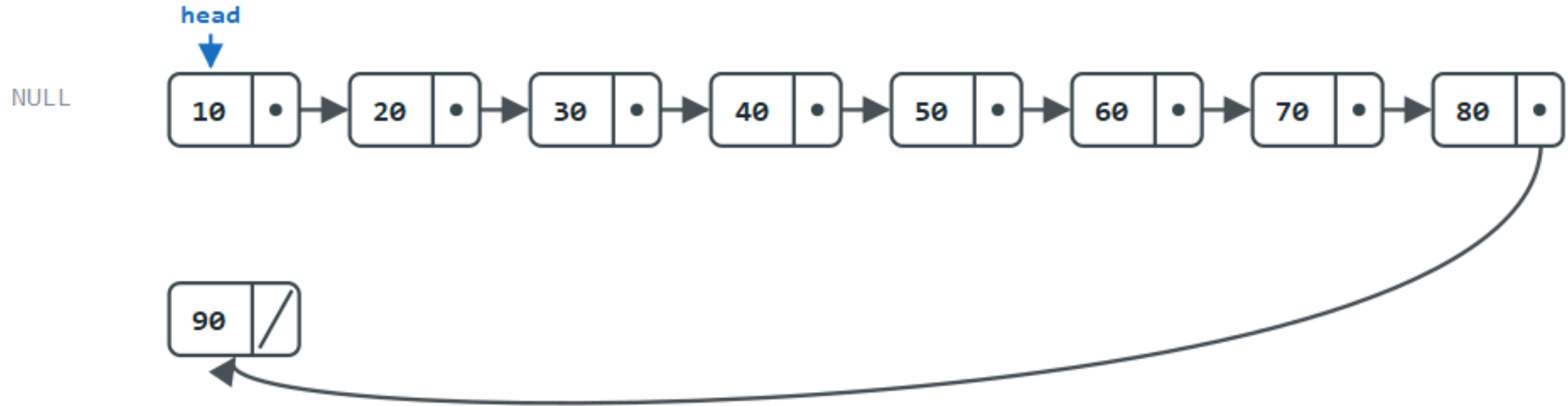
- Özel durum: değer **başta**
- Aksi halde: `prev` / `cur` birlikte yürüyerek arar
- Bulunduğunda: `prev->next = cur->next` — **bypass oku**
- Sonra `free(cur)` — düğüm gitti

Tekil listede silme, adım adım



Bitti: 2 silme başarılı, 0 bulunamadı. Liste (baştan sona): 20, 30, 40, 50, 70, 80, 90, 100.

Uç durum — son düğümü, sonra olmayan bir değeri silmek



Bitti: 1 silme başarılı, 1 bulunamadı. Liste (baştan sona): 10, 20, 30, 40, 50, 60, 70, 80, 90.

Kod — delete_value()

```
Node *delete_value(Node *head, int value,
                    bool *removed) {
    *removed = false;
    if (head == NULL) return NULL;
    if (head->data == value) { /* delete head */
        Node *tmp = head;
        head = head->next;
        free(tmp);
        *removed = true;
        return head;
    }
    /* ... prev/cur scan, bypass, free(cur) ... */
}
```

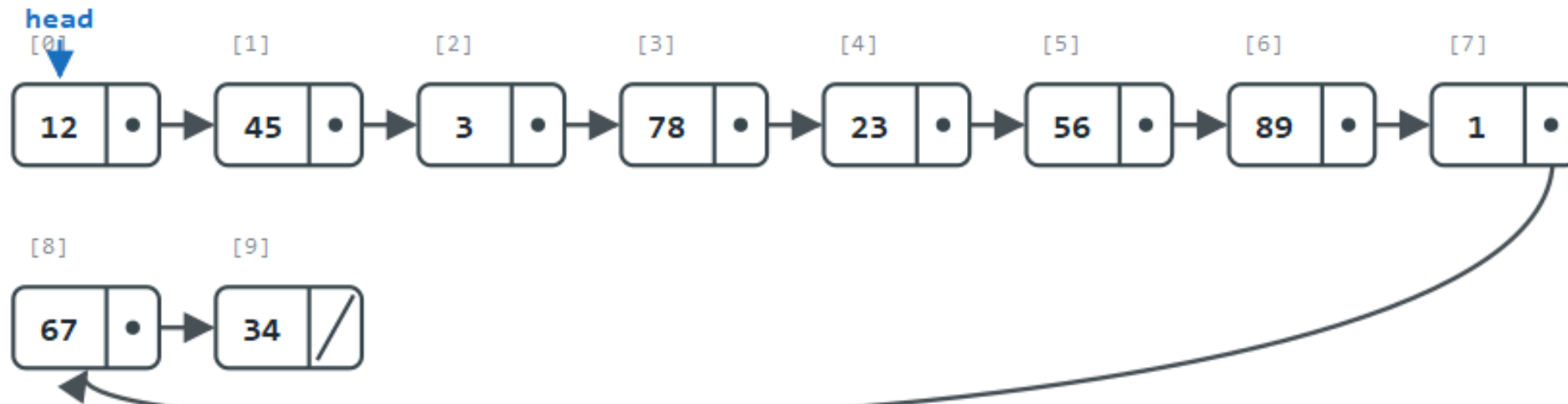
Karmaşıklık ve sık yapılan hatalar

- Başı silmek: $O(1)$, arama gerekmez
- Başka herhangi bir yeri silmek: bulmak için $O(n)$
- Hata: başın **özel** bir durum olduğunu unutmak
- Hata: `prev` 'i kaydetmeden `cur` 'u ilerletmek

Doğrusal arama: kısayol yok

- İndis aritmetiği yok — bir listede `arr[k]` yoktur
- `head` 'den başlayıp bir seferde bir düğümü karşılaştır
- Bulundu: **konumunu** döndür; bitti: `-1` döndür

Tekil listede arama, adım adım



Bitti: 1 arama. Sonuçlar (konum, karşılaştırma): (4, 5).

Uç durum — boş bir listede arama



Bitti: 1 arama. Sonuçlar (konum, karşılaştırma): (-1, 0).

Kod — search()

```
int search(Node *head, int value) {  
    int index = 0;  
    for (Node *cur = head; cur != NULL;  
         cur = cur->next) {  
        if (cur->data == value)  
            return index;    /* found here */  
        index++;  
    }  
    return -1;                /* not found */  
}
```

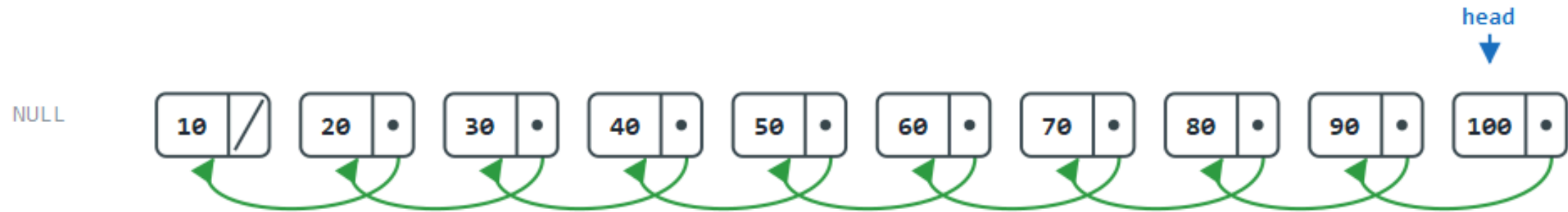
Karmaşıklık

- Her arama: $O(n)$ — atlanacak bir indis yok
- Karşılaştırma: dizide `arr[k]` $O(1)$ 'dir
- Bu fark, üç bölüm sonraki karşılaştırmayı yönlendirir

Yerinde tersine çevirme: üç işaretçi

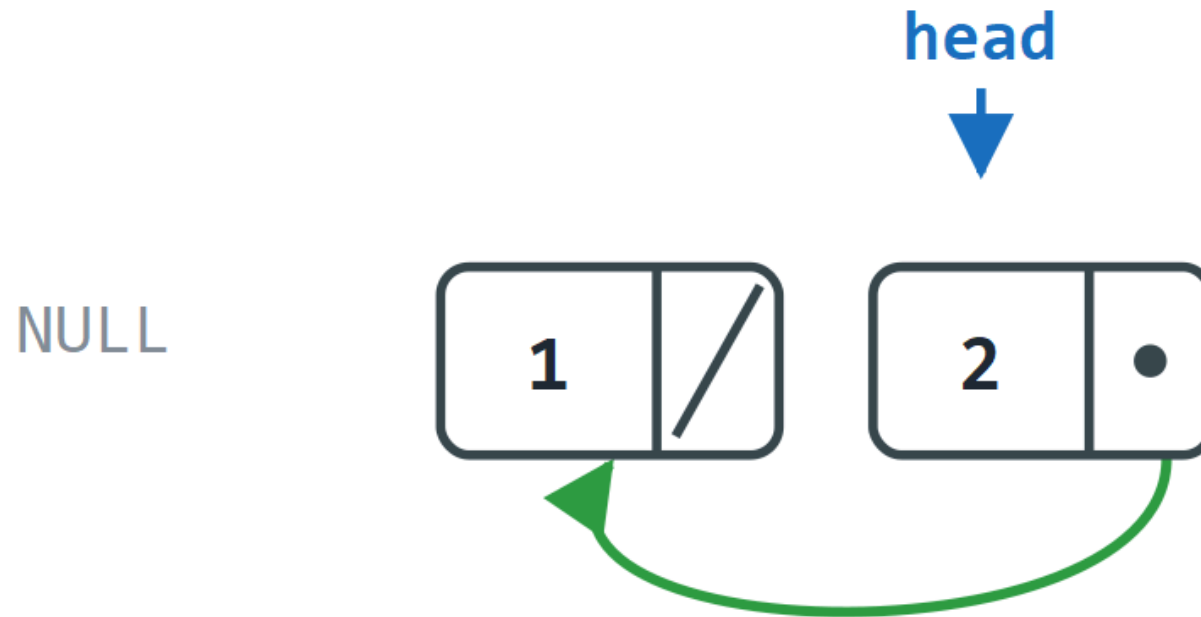
- `prev` arkada, `curr` önde, `next` ileriye bakar
- `curr->next` 'in üzerine yazmadan **önce** `next` 'i kaydet
- `curr->next` 'i çevirip `prev` 'i gösterecek şekilde yap
- Sonra her iki işaretçi de bir düğüm ileri gider

Tekil listede tersine çevirme, adım adım



`curr == NULL`: döngü bitti. `prev`, yeni baş olan eski son düğümü gösteriyor; `head` de artık onu gösteriyor.

Uç durum — yalnızca iki düğüm



`curr == NULL`: döngü bitti. prev, yeni baş olan eski son düğümü gösteriyor; head de artık onu gösteriyor.

Kod — reverse()

```
Node *reverse(Node *head) {  
    Node *prev = NULL;  
    Node *curr = head;  
    while (curr != NULL) {  
        Node *next = curr->next; /* save rest */  
        curr->next = prev;      /* flip arrow */  
        prev = curr;  
        curr = next;  
    }  
    return prev;                /* new head */  
}
```

Karmaşıklık

- Tek geçiş, düğüm başına bir çevirme — $O(n)$ zaman
- Yeni düğüm ayrılmıyor — $O(1)$ alan
- Karşılaştırma: bir diziyi tersine çevirmek de ekstra işaretçi gerektirmez

Mini soru

`curr->next` 'i geçici bir `next` değişkenine kaydetmeden **önce** `prev` 'i göstermek üzere çevirirseniz ne bozulur?

Yanıt

Listenin geri kalanı kaybolur. `curr->next`

geriye işaret ettiği an, ondan sonra gelen düğümlere ulaşmanın hiçbir yolu kalmaz.

7. Çift Yönlü Bağlı Listeler

Bir soru ile başlayalım

Tekil bir liste yalnızca **ileri** yürüyebilir.
Geriye de yürüyebilmek için ne değişmeli?

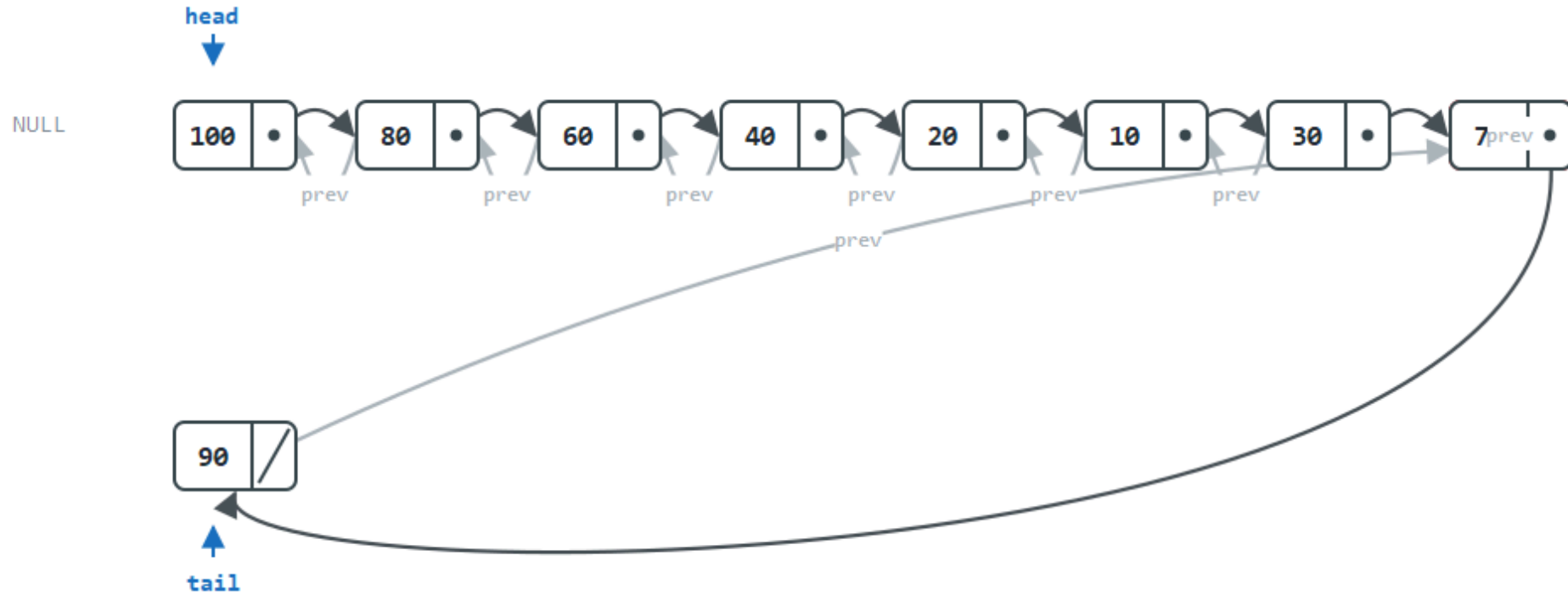
Sezgi — çift yönlü bir cadde

- Tekil liste: tek yönlü bir cadde, yalnızca ileri
- Çift yönlü liste: çift yönlü bir cadde
- Her düğüm **iki** tabela taşır: `next` ve `prev`
- Geri dönmek hiçbir ek bedele mal olmaz — yalnızca `prev` 'i izleyin

`insert_after` : ikisi değil, dört işaretçi düzeltilir

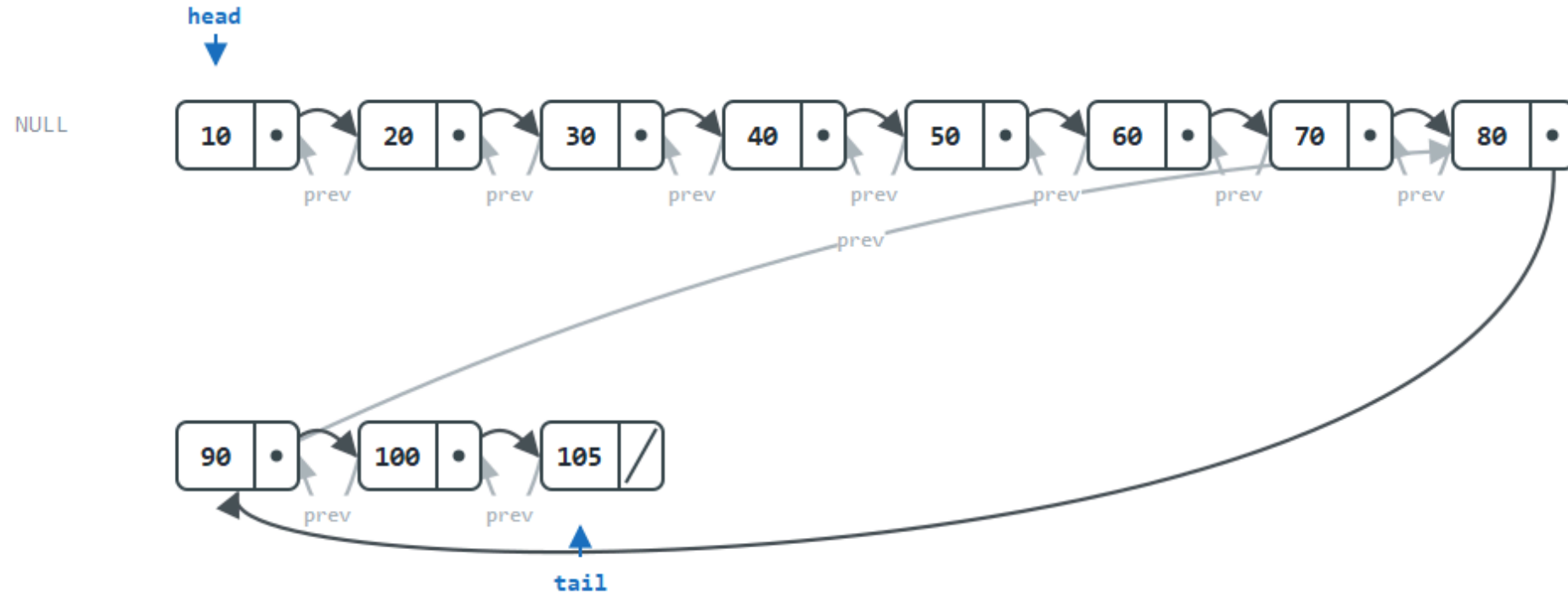
- Tekil listenin `insert_after` 'ı: 2 işaretçiyi düzeltir
- Çift yönlü liste: **yeni** düğümün `prev` 'ini de düzeltir
- Ve **eski sonrakinin** `prev` 'ini, varsa
- `cur`, `tail` idiyse `list->tail` 'i de güncelleyin

Çift yönlü listede ekleme, adım adım



Bitti: 10 ekleme, 1 silme (0 bulunamadı). İleri: 100, 80, 60, 40, 20, 10, 30, 70, 90. Geri: 90, 70, 30, 10, 20, 40, 60, 80, 100.

Uç durum — tail'in gösterdiği düğümden hemen sonra ekleme



Bitti: 11 ekleme, 0 silme (0 bulunamadı). İleri: 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 105. Geri: 105, 100, 90, 80, 70, 60, 50, 40, 30, 20, 10.

Kod — insert_after()

```
bool insert_after(List *list, int target, int v) {
    for (Node *cur = list->head; cur; cur = cur->next) {
        if (cur->data == target) {
            Node *n = malloc(sizeof(Node));
            n->data = v; n->prev = cur; n->next = cur->next;
            if (cur->next) cur->next->prev = n;
            else list->tail = n; /* cur was the tail */
            cur->next = n;
            return true;
        }
    }
    return false;
}
```

Kod — `delete_value()` : `prev` için tarama yok

```
bool delete_value(List *list, int value) {  
    for (Node *cur = list->head; cur; cur = cur->next) {  
        if (cur->data == value) {  
            if (cur->prev) cur->prev->next = cur->next;  
            else list->head = cur->next;  
            if (cur->next) cur->next->prev = cur->prev;  
            else list->tail = cur->prev;  
            free(cur);  
            return true;  
        }  
    }  
    return false;  
}
```

Karmaşıklık ve sık yapılan hatalar

- Hedefi bulmak: tekildekiyle aynı $O(n)$
- Bulunduktan sonra yeniden bağlamak: her iki yönde $O(1)$
- Hata: tail değiştikten sonra `tail` 'i güncellemeyi unutmak
- Hata: yeni düğümün `prev` bağını atlamak

Mini soru

Çift yönlü bir listenin `delete_value` 'u,
tekil sürümün aksine, neden ayrı bir
`prev` -izleme değişkenine hiç ihtiyaç duymaz?

Yanıt

Her düğüm zaten kendi `prev` 'ini saklar.

Tekil sürüm tararken `prev` 'i elle takip etmek zorundaydı; çift yönlü sürüm yalnızca `cur->prev` 'i doğrudan okur.

8. Dairesel Listeler ve Josephus Problemi

Bir soru ile başlayalım

Ya **son** düğümün `next` i, `NULL` yerine ilk düğümü gösterseydi?

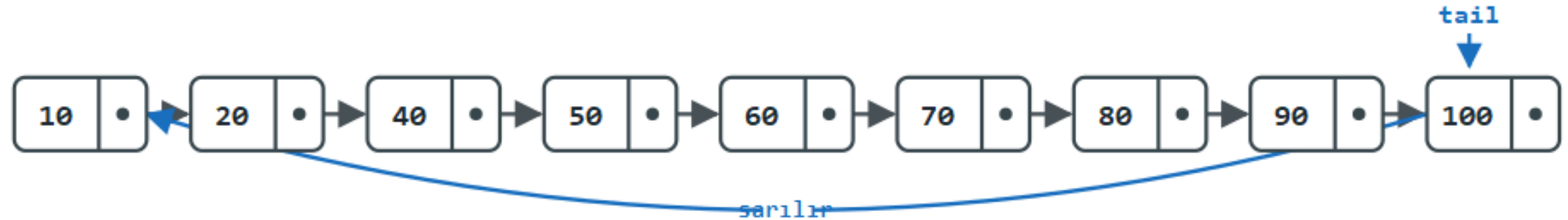
Sezgi — bir çemberde oturan insanlar

- Aslında ne "ilk" koltuk ne de "son" koltuk var
- Yeterince yürürseniz başladığınız yere geri dönersiniz
- Dairesel bir listenin yürüyüşü durduracak bir `NULL` 'ı yok
- Başka bir şey — bir sayaç, bir koşul — durdurmalı

Tek işaretçi, ayrı bir `head` yok

- `tail`, en son eklenen düğümü gösterir
- `tail->next`, **baştır** — ekstra bir alan yok
- Tek bir düğüm **kendini** gösterir — küçük bir döngü
- Silmek ileri bir tarama ister — `prev` yok

Dairesel liste işlemleri, adım adım



Bitti: 2 tur, toplam 18 adım (9 düğüm x 2), 1 silme (0 bulunamadı). Gezinme: 10, 20, 40, 50, 60, 70, 80, 90, 100, 10, 20, 40, 50, 60, 70, 80, 90, 100.

Uç durum — son düğüm silinir

(boş)

`traverse(NULL, 5): gösterilecek düğüm yok.`

Kod — `insert_tail()`

```
Node *insert_tail(Node *tail, int value) {
    Node *n = malloc(sizeof(Node));
    n->data = value;
    if (tail == NULL) {
        n->next = n;      /* points at itself */
        return n;
    }
    n->next = tail->next; /* new node -> old head */
    tail->next = n;       /* old tail -> new node */
    return n;
}
```

Karmaşıklık ve sık yapılan hatalar

- Sona ekleme: $O(1)$, öncekiyle aynı numara
- Değere göre silme: $O(n)$ tarama — `prev` bağı yok
- Hata: bir gezinmeyi durdurmak için `cur == NULL` kontrolü
- Dairesel bir liste bir `NULL` değil, bir **adım sayacı** ister

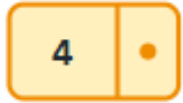
Kısa bir tarihçe

- **M.S. 67 civarı** — Flavius Josephus, Yodfat kuşatması
- Efsane: 41 asker, her 3. asker elenir
- Josephus'un kendini hayatta kalan yere yerleştirdiği söylenir
- Tek bir dairesel liste bugün tüm bulmacayı çözüyor

Her k . kişi elenir

- Çember içinde n kişi, 1.. n numaralı
- Son hayatta kalandan $k-1$ adım ileri say
- Üzerine geldiğiniz kişiyi eleyin
- Yalnızca **bir** kişi kalana kadar tekrarlayın

Josephus problemi, adım adım



Bitti: hayatta kalan kişi 4. Eleme sırası: 3, 6, 9, 2, 7, 1, 8, 5, 10. (Not: kapalı biçim -- $J(1)=0$, $J(n)=(J(n-1)+k) \bmod n$ -- her adımı simüle etmeden aynı hayatta kalanı verir.)

Uç durum — tek kişilik bir çember



$n = 1$: çember bir kişiden ibaret, $\text{remaining} > 1$ hiç doğru olmuyor -- döngü çalışmadan $\text{cur} \rightarrow \text{id}$ (1) döner.

Kod — josephus()

```
int josephus(int n, int k) {
    Node *cur = head;
    int remaining = n;
    while (remaining > 1) {
        for (int s = 1; s < k; s++) { /* k-1 steps */
            prev = cur;
            cur = cur->next;
        }
        prev->next = cur->next; /* remove cur */
        cur = prev->next;
        remaining--;
    }
    return cur->id; /* the survivor */
}
```

Karmaşıklık

- Doğrudan simüle etmek: en kötü durumda $O(n \cdot k)$
- Kapalı biçimli bir yineleme, hayatta kalanı $O(n)$ 'de verir
- $J(1)=0$, $J(n) = (J(n-1) + k) \bmod n$

Mini soru

$k = 1$ iken her sayım yalnızca "bir sonraki kişi". 10 kişilik bir çemberde kim hayatta kalır?

Yanıt

10. kişi — sıradaki son kişi. $k=1$

ile hiç atlama yok: eleme yalnızca düz
sırayla ilerler, 1'den 9'a kadar.

9. XOR Bağlı Listeler: Bir Merak Konusu

Bir soru ile başlayalım

Çift yönlü bir liste düğüm başına iki işaretçi alanı harcar. **Tek** bir alan bir şekilde iki komşuyu birden tutabilir mi?

$$\text{npx} = \text{prev} \text{ XOR } \text{next}$$

- **Tek** alan sakla: `prev` 'in adresi XOR `next` 'in adresi
- Bilinen bir komşu `known` 'dan gelindiğinde diğerini kur:
- `other = npx XOR known`
- XOR, geldiğiniz komşuyu sessizce "iptal" eder

XOR listede gezinme, adım adım

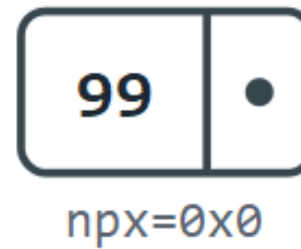
Her düğümün YALNIZ bir alanı var: `npx = prev XOR next`.



Bitti. İleri: 10, 20, 30, 40, 50, 60, 70, 80, 90, 100. Geri: 100, 90, 80, 70, 60, 50, 40, 30, 20, 10. Tek bir npx alanıyla iki yönlü gezinme -- bedeli her adımda bir XOR.

Uç durum — boş bir listenin sonuna ekleme

Her düğümün YALNIZ bir alanı var: ``npx = prev XOR next``.



Bitti. İleri: 99. Geri: 99. Tek bir npx alanıyla iki yönlü gezinme -- bedeli her adımda bir XOR.

Kod — struct ve XOR numarası

```
typedef struct Node {  
    int data;  
    uintptr_t npx;    /* XOR of prev and next */  
} Node;  
  
static Node *xor_node(uintptr_t npx, Node *known) {  
    return (Node *) (npx ^ (uintptr_t) known);  
}
```

Kod — `traverse_forward()`

```
void traverse_forward(Node *head) {  
    Node *prev = NULL, *cur = head;  
    while (cur != NULL) {  
        printf(" %d", cur->data);  
        Node *next = xor_node(cur->npx, prev);  
        prev = cur; cur = next;  
    }  
}
```

Karmaşıklık

- Başa veya sona ekleme: öncekiyle aynı **$O(1)$**
- Her gezinme adımı: bir fazladan **XOR** — yine de $O(1)$ her biri
- Tasarruf edilen bellek: düğüm başına bir işaretçi alanı

Bunun neden bir merak konusu olduğu, alışkanlık değil

- C, işaretçi↔tamsayı gidiş-dönüşlerini bu şekilde **garanti etmez**
- `uintptr_t` 'i işaretçiye geri çevirmek **derleyiciye bağlıdır**
- Çöp toplayıcılı bir dil (Java) bunu hiç yapamaz
- Gerçek kod: bunun yerine düz bir çift yönlü liste kullanın

Mini soru

Bir çöp toplayıcı, C'nin `malloc` 'unun yaptığı gibi bir XOR bağlı listeyi neden hiç destekleyemez?

Yanıt

Bir GC, izlemek ve gerekirse taşımak için her canlı işaretçiyi bulmalıdır. XOR'lanmış bir tamsayının içine gizlenmiş bir adres, o taramaya görünmezdir — GC onu bir işaretçi olarak göremez.

10. Atlamalı Listeler

Bir soru ile başlayalım

Sıralı bir dizi ikili arama alır, $O(\log n)$.
Sıralı bir bağlı liste ortaya sıçrayamaz.
Yine de $O(\log n)$ arama mümkün mü?

Kısa bir tarihçe

- **1990** — William Pugh, Maryland Üniversitesi
- Dengeli arama ağaçlarına rastgele bir alternatif
- Her anahtar rastgele bir "yazı-tura" yüksekliği alır
- Doğru uygulamak, dengeli bir ağaçtan çok daha kolay

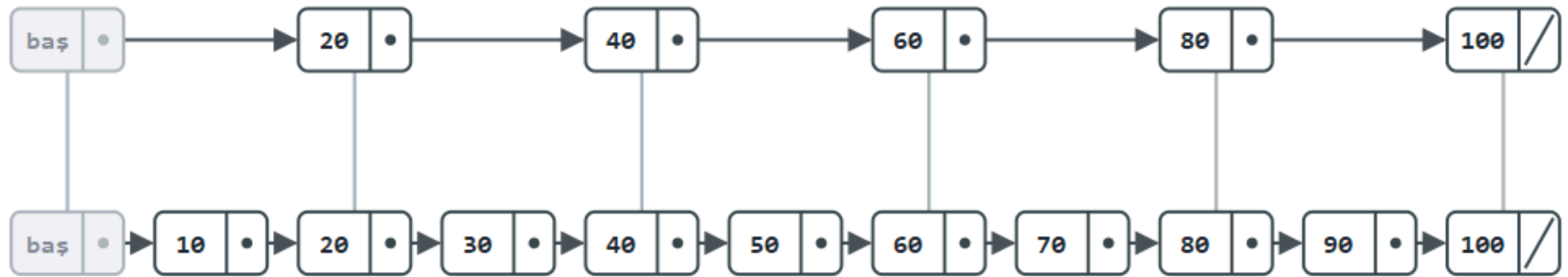
Sezgi — bir ara yol ve bir hızlı şerit

- Seviye 0: tam sıralı liste — her anahtar
- Seviye 1: bir **hızlı şerit** — yalnızca bazı anahtarlar
- Hızlı şeritte başlayın; fazla ileri gidince **aşağı** **inin**
- Her anahtarı teker teker kontrol etmekten daha az durak

Arama: sağa git, ya da aşağı in

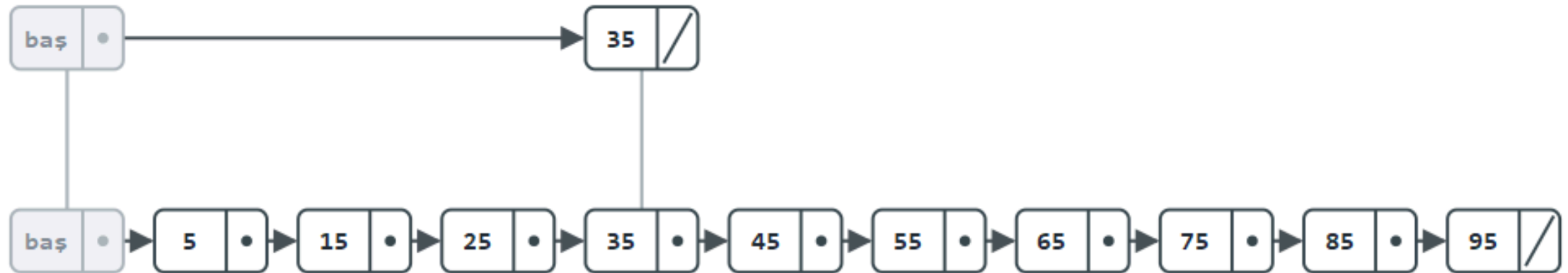
- Şu anki seviyede, `cur->forward[i]->value < hedef` mi?
- Evet: bu seviyede kalarak sağa adım at
- Hayır: bir seviye **aşağı in** ve yeniden dene
- Seviye 0'a ulaşıp bir kez daha adım atmak: yanıt

Atlama listesi araması, adım adım



Bitti: 10 anahtar, 2 arama. Taban liste (sıralı): 10, 20, 30, 40, 50, 60, 70, 80, 90, 100.

Uç durum — hızlı şeritte yalnızca bir düğüm



Bitti: 10 anahtar, 2 arama. Taban liste (sıralı): 5, 15, 25, 35, 45, 55, 65, 75, 85, 95.

Kod — `sl_search()`

```
int sl_search(SkipList *sl, int value, int *cmp) {
    Node *cur = sl->header;
    for (int i = MAX_LEVEL-1; i >= 0; i--) {
        while (cur->forward[i] &&
               cur->forward[i]->value < value) {
            cur = cur->forward[i];    /* go right */
            (*cmp)++;
        }
        /* ... else: drop down one level ... */
    }
    cur = cur->forward[0];
    return cur && cur->value == value;
}
```

Karmaşıklık ve sık yapılan hatalar

- Yeterli hızlı seviye: $O(\log n)$ beklenen arama
- Her anahtar yalnızca seviye 0'da: $O(n)$ 'ye düşer
- Hata: bir seviye aşağı inmek yerine durmak
- Bu demonun seviyeleri sabit, gerçekten canlı atılmış değil

Mini soru

Bir atlamalı listedeki her anahtar yalnızca seviye 1'i alsaydı (hiç hızlı şerit olmasaydı), arama Big-O olarak ne tutar?

Yanıt

$O(n)$. Hiç hızlı şerit yokken, seviye 0 kalan tek seviyedir — arama düz bir bağlı listeye aynı doğrusal taramaya düşer.

11. Diziler ve Bağlı Listeler

Diziler ve bağlı listeler

İşlem	Dizi	Bağlı liste
İndise erişim	$O(1)$	$O(n)$
Baştan ekleme/silme	$O(n)$	$O(1)$
Değere göre arama	$O(n)$	$O(n)$

Hangisini ne zaman seçmeli

- Sürekli `arr[k]` mi gerekiyor? Diziler açık ara kazanır
- Sık baştan ekleme, bilinmeyen son boyut? Listeler kazanır
- Önbellek dostu, bitişik taramalar? Diziler kazanır
- Eklerken var olan veriyi hiç kaydırmamak? Listeler kazanır

Mini soru

Yalnızca bir uçtan büyüyüp küçülen bir yığın (stack) kuruyorsunuz. Dizi mi, liste mi?

Yanıt

İkisi de iyi çalışır — ikisi de bir uçta $O(1)$ verir: bir dinamik dizinin `append` 'i, ya da bir listenin `insert_head` 'i. Asıl belirleyici, ayrıca `arr[k]` 'ye ihtiyaç duyup duymadığınız.

Özet — diziler ve matrisler

Fikir	Anahtar olgu
Dizide ekleme/silme	Kaydırma; en kötü $O(n)$, sonda $O(1)$
Dinamik dizi	İkiye katlama; $O(1)$ amortize ekleme
Satır öncelikli yerleşim	$i * COLS + j$; gezinmeyi yerleşime eşle
Seyrek matris	Triplet'ler; hızlı devrik $O(nnz + COLS)$

Özet — bağlı listeler

Fikir	Anahtar olgu
Tekil liste	insert_head $O(1)$; insert_after: sıra önemli
Çift yönlü / dairesel	İki bağ; tail->next baştır
Josephus	Dairesel liste, eleme başına bir bypass
XOR / atlamalı liste	Tek-alan numarası; rastgele $O(\log n)$

Büyük resim

Diziler, esnek eklemeyi $O(1)$ indisleme karşılığında feda eder; bağlı listeler, indislemeyi zaten bulduğunuz her yerde $O(1)$ ekleme karşılığında feda eder. Bu haftaki her yapı, aynı ödünleşimin bir tarafını seçiyor.

Kendi kendine kontrol turu

Dört kısa soru. Yanıt bir sonraki slaytta görünmeden önce düşünün. Tam alıştırmalar hafta notunda.

1. 20 elemanlı bir dizide 0. indise eklemek: kaç eleman hareket eder?

Tam 20'si — dizinin tamamı bir hücre sağa kayar.

2. Büyümesi $O(n)$ 'e mal oluyorsa, bir dinamik dizinin eklemesi neden hâlâ $O(1)$ deniyor?

$O(n)$ büyüme nadirdir, ve bedeli **amortize olur**: n ekleme üzerinden ortalandığında toplam $2n$ 'nin altında kalır.

3. `insert_after` 'da `n->next = prev->next` , neden `prev->next = n` 'den önce olmalı?

Ters çevrilirse, 1. adım onu okuduğunda `prev->next` zaten `n`'e eşittir — yeni düğüm kendini gösterir hale gelir.

4. Bir atlamalı listenin arama maliyeti, hızlı şerit olmadığında neden $O(n)$ 'e düşer?

Üst seviye yokken, seviye 0 kalan tek seviyedir — arama düz bir bağlı listeye aynı, bir seferde bir anahtarlık yürüyüşe dönüşür.

Gelecek hafta

Hafta 3 — Yığınlar ve Kuyruklar

LIFO ve FIFO, bu haftanın dizi ve bağlı liste araçları üzerine doğrudan kurulacak — aynı düğümler, aynı kaydıran dizi, artık tek bir giriş/çıkış noktasına disiplinlenmiş.

Kaynaklar (1/2)

- Ders izlencesi, Hafta 2: `docs/syllabus/syllabus.tr.md`
- Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 4. baskı. MIT Press
- Sedgewick, Wayne. *Algorithms*, 4. baskı. Addison-Wesley
- Knuth. *The Art of Computer Programming, Cilt 1*, 3. baskı.

Kaynaklar (2/2)

- Pugh (1990) — atlamalı listeler
- McCarthy (1956) — Lisp cons hücreleri
- Flavius Josephus, *The Jewish War* — eleme problemi
- [williamfiset/Algorithms](#) · Programiz DSA